WFLOW-08: JavaScript & HTTP Actions

> **Series:** WFLOW — Workflows and Alert Notifications | **Notebook:** 8 of 10 | **Created:** January 2026 | **Last Updated:** 10/27/2026

Custom Code and API Integration

When built-in actions aren't enough, use JavaScript and HTTP requests for custom integrations. This notebook covers the JavaScript SDK, HTTP request patterns, and common integration scenarios.

Table of Contents

1. JavaScript Action Basics
2. Dynatrace SDK Usage
3. HTTP Request Patterns
4. Error Handling
5. Workflow-Level Failure Branching
6. Working with Data
7. Passing Data Between Tasks
8. Integration Examples
9. Performance Tips
10. Configuring Task Timeouts
11. CMDB-Driven Host Tag Enrichment

Prerequisites

Requirement	Details
Dynatrace Environment	SaaS with Platform subscription

Requirement	Details
Permissions	`automation:workflows:write` to author. Each task also needs the scopes its SDK calls use, granted to the workflow under **Settings > Authorization settings** in the Workflows app — for example `automation:workflows:read` for `execution()` and `result()`, `environment-api:credentials:read` for the Credential Vault, `environment-api:entities:read` for entity lookups, `environment-api:problems:write` for problem comments, and the `storage:*:read` scope of each data object a query reads. A missing one fails the task with a 403. §11's workflow actor also needs `fleet-management:oneagents:write` (a classic `oneAgents.write` token is the classic/hybrid alternative)
Outbound access	Every external host added under **Settings > General > External requests**; hosts on a private network also need an EdgeConnect host pattern (§3)
Prior Knowledge	**WFLOW-01** through **WFLOW-07**, JavaScript basics

> **Sources:** Workflows security (DT docs) — *"If the required permission for a workflow task is missing, an attempt to execute this task results in a 403 Forbidden error."*

1. JavaScript Action Basics

Action Structure

```
// Every JavaScript action must export a default async function.
// The runtime injects only identifiers: { executionId, actionExecutionId } (plus loopItemValue
// in a loop). Event data, predecessor results and secrets come from SDK imports, not
parameters.
export default async function ({ executionId, actionExecutionId }) {
  // Your logic here
  const result = await doSomething();

  // Return value becomes task result
  return {
    output: result,
    success: true
  };
}
```

Accessing Context

```
import { execution, result } from '@dynatrace-sdk/automation-utils';
import { credentialVaultClient } from '@dynatrace-sdk/client-classic-environment-v2';

export default async function ({ executionId, actionExecutionId }) {
  const ev = (await execution()).params.event;           // trigger payload
  const prev = await result('previous_task');              // predecessor result (async)
  const cred = await credentialVaultClient.getCredentialsDetails({ id: 'CREDENTIALS_VAULT-
XXXXXXXXXXX' });
  // use cred.token in a request header; never return the credential
  return { title: ev['event.name'] };
}
```

There is no `env` object and no `event` parameter: *"The runtime injects workflow and task identifiers into the function parameters"*, and *"For workflows triggered by an event, retrieve the event payload from the execution"*. Secrets come from the Credential Vault: *"To use a Credential Vault secret in a script, for example, as an API key in an HTTP request, retrieve it with credentialVaultClient"* (Run JavaScript action (DT docs)). The credential needs the AppEngine scope and **Allow access without app context** turned on, and the workflow actor needs access to it.

Task Configuration

```
name: custom_javascript
action: dynatrace.automations:run-javascript
input:
  script: |
    export default async function () {
      return { processed: true };
    }
```

2. Dynatrace SDK Usage

Available SDK Clients

Client	Purpose	Import
`queryExecutionClient`	Execute DQL queries	`@dynatrace-sdk/client-query`
`monitoredEntitiesClient`	Read monitored entities	`@dynatrace-sdk/client-classic-environment-v2`
`problemsClient`	Problem management	`@dynatrace-sdk/client-classic-environment-v2`
`eventsClient`	Event ingestion	`@dynatrace-sdk/client-classic-environment-v2`

Client	Purpose	Import
`settingsObjectsClient`	Settings 2.0 objects	`@dynatrace-sdk/client-classic-environment-v2`

Execute DQL Query

`queryExecute()` does not always return the result. It returns it only if the query finishes within `requestTimeoutMilliseconds`; otherwise the response carries a `requestToken` instead, and reading `result.result.records` throws. Wrap every query in a helper that polls until the query reaches a final state — the rest of this notebook uses this `runQuery()`:

```
import { queryExecutionClient } from '@dynatrace-sdk/client-query';

// queryExecute() returns the result only if the query finishes within
// requestTimeoutMilliseconds. Otherwise it returns a requestToken and a state
// of NOT_STARTED or RUNNING, and the result has to be fetched with queryPoll().
async function runQuery(query) {
  const started = await queryExecutionClient.queryExecute({
    body: { query, requestTimeoutMilliseconds: 30000 }
  });
  let response = started;
  while (response &amp;&amp; (response.state === 'NOT_STARTED' || response.state ===
'RUNNING')) {
    response = await queryExecutionClient.queryPoll({
      requestToken: started.requestToken,   // poll responses carry no token of their own
      requestTimeoutMilliseconds: 30000
    });
  }
  if (!response || response.state !== 'SUCCEEDED') {
    throw new Error(`DQL query did not succeed: ${response?.state ?? 'no response'}`);
  }
  return response.result.records;
}

export default async function () {
  const records = await runQuery(`
    fetch logs, from: now() - 1h
    | filter status == "ERROR"
    | summarize error_count = count()
  `);

  return { error_count: records[0]?.error_count ?? 0 };
}
```

The polling loop is bounded by the runtime's own per-action ceiling (§10): a query that cannot finish inside it fails the task either way, so the fix for a slow query is a narrower query, not a longer loop.

> **Sources:** client-query SDK (Dynatrace Developer) — `queryExecute` `requestTimeoutMilliseconds`: *"If the query finishes within the specified timeout, the query result is returned. Otherwise, the requestToken is returned, allowing polling for*

> *the result."*; `queryPoll` : *"Retrieves query status and final result from Grail."*; state values *"NOT_STARTED"*, *"RUNNING"*, *"SUCCEEDED"*, *"RESULT_GONE"*, *"CANCELLED"*, *"FAILED"* (re-read 09/28/2026).

Get Entity Details

```
import { execution } from '@dynatrace-sdk/automation-utils';
import { monitoredEntitiesClient } from '@dynatrace-sdk/client-classic-environment-v2';

export default async function () {
  const ev = (await execution()).params.event;
  const entityId = ev.root_cause_entity_id;   // classic entity ID; absent on most problems
  if (!entityId) {
    return { skipped: 'problem has no root-cause entity' };
  }

  const entity = await monitoredEntitiesClient.getEntity({
    entityId: entityId,
    fields: '+properties,+tags'
  });

  return {
    name: entity.displayName,
    type: entity.type,
    tags: entity.tags || []
  };
}
```

> **Most problems name no root cause.** On the validation tenant over seven days (10/06/2026), 3,348 of 4,241 problems carried neither `root_cause_entity_id` nor its replacement `root_cause.smartscape_entity`, so check the field before you use it. The upgrade guide states that *"root_cause_entity_id and root_cause_entity_name are deprecated in favor of root_cause.smartscape_entity, and affected_entity_ids in favor of smartscape.affected_entities."* `getEntity` takes a classic entity ID; WFLOW-07 §1 has a helper that reads the Smartscape field first and falls back to the classic one.
>
> **Sources:** Upgrade guide: alert notifications (DT docs) — *"Dynatrace Intelligence does not populate a root cause for every problem, particularly early in the lifecycle or for externally ingested events."*; client-classic-environment-v2 SDK (Dynatrace Developer).

Add Problem Comment

> **Deprecation — Dynatrace API 1.349 (staged tenant rollout from 10/27/2026).** The API 1.349 changelog (DT docs) marks the Environment API v2 `/problems` endpoints deprecated, including *"POST /problems/{problemId}/comments Deprecated"*, and names no successor. Verify it has reached your tenant; until then the current guidance remains the working path.

```
import { execution, result } from '@dynatrace-sdk/automation-utils';
import { problemsClient } from '@dynatrace-sdk/client-classic-environment-v2';

export default async function () {
  const ev = (await execution()).params.event;
  const problemId = ev['event.id'];                        // the problem ID ({PID}), not
display_id
  const actionTaken = (await result('remediation')).action;

  await problemsClient.createComment({
    problemId: problemId,
    body: {
      message: `[Automated] Remediation executed: ${actionTaken}`,
      context: 'Workflow automation'
    }
  });

  return { comment_added: true };
}
```

3. HTTP Request Patterns

Allow the Host First

Every host a task calls must be allow-listed. For a script, *"The external endpoint must be added to External requests."* For the HTTP Request action, *"All HTTP calls are validated against the global allowlist."* Add the host under **Settings > General > External requests**. The workflow runs in Dynatrace SaaS, so a host on your private network also needs an EdgeConnect host pattern: *"Any HTTP request (from your app functions, workflows, or ad-hoc functions) that matches a defined host pattern is handled by an EdgeConnect instance"*. The **WFLOW-94 LAB** sets one up.

> **Sources:** Run JavaScript action (DT docs), HTTP request action (DT docs), EdgeConnect (DT docs) — *"Use EdgeConnect to make apps and workflows interact securely with your systems."*

HTTP Request Action

```
name: call_external_api
action: dynatrace.automations:http-function
input:
  url: "https://api.example.com/endpoint"
  method: POST
  # Authentication: select a Credential Vault token in the task's Authentication field
  headers:
    Content-Type: "application/json"
  payload: |                # the request body: the action's Payload field
    {
      "problem_id": {{ event()['display_id'] | to_json }},
      "category": {{ event()['event.category'] | to_json }},
      "title": {{ event()['event.name'] | to_json }},
      "link": {{ problem_link() | to_json }}
    }
```

Each value goes through `to_json`, which *"Converts an object to its JSON representation"*, so a quote or backslash in a problem title cannot break the JSON. The request body goes in `payload`; the action has no `body` input (*"Payload : The payload of the HTTP request."*). The HTTP Request action docs also say: *"We strictly advise against providing any static Authorization header and therefore, leak a secret. Use the credential vault to store your credentials for Basic or Token authentication"* (HTTP request action (DT docs)).

Using fetch() in JavaScript

```
import { execution } from '@dynatrace-sdk/automation-utils';
import { credentialVaultClient } from '@dynatrace-sdk/client-classic-environment-v2';

export default async function () {
  const ev = (await execution()).params.event;   // trigger payload
  const token = (await credentialVaultClient.getCredentialsDetails({ id: 'CREDENTIALS_VAULT-
XXXXXXXXXXX' })).token;
  const response = await fetch('https://api.example.com/webhook', {
    method: 'POST',
    headers: {
      'Authorization': `Bearer ${token}`,
      'Content-Type': 'application/json'
    },
    body: JSON.stringify({
      problem_id: ev.display_id,
      category: ev['event.category'],
      title: ev['event.name'],
      timestamp: new Date().toISOString()
    })
  });

  if (!response.ok) {
    throw new Error(`HTTP ${response.status}: ${await response.text()}`);
  }

  const data = await response.json();
  return { response: data };
}
```

GET Request with Query Parameters

```
import { execution } from '@dynatrace-sdk/automation-utils';
import { credentialVaultClient } from '@dynatrace-sdk/client-classic-environment-v2';

export default async function () {
  const ev = (await execution()).params.event;   // trigger payload
  if (!ev.root_cause_entity_id) {
    return { skipped: 'problem has no root-cause entity' };
  }
  const token = (await credentialVaultClient.getCredentialsDetails({ id: 'CREDENTIALS_VAULT-
XXXXXXXXXXX' })).token;
  const params = new URLSearchParams({
    entity_id: ev.root_cause_entity_id,
    from: new Date(Date.now() - 3600000).toISOString(),
    to: new Date().toISOString()
  });

  const response = await fetch(
    `https://api.example.com/metrics?${params}`,
    {
      headers: {
        'Authorization': `Bearer ${token}`
      }
    }
  );

  return await response.json();
}
```

4. Error Handling

Try-Catch Pattern

```
import { execution } from '@dynatrace-sdk/automation-utils';
import { credentialVaultClient } from '@dynatrace-sdk/client-classic-environment-v2';

export default async function () {
  try {
    const ev = (await execution()).params.event;   // trigger payload
    const token = (await credentialVaultClient.getCredentialsDetails({ id: 'CREDENTIALS_VAULT-
XXXXXXXXXXXX' })).token;
    const response = await fetch('https://api.example.com/action', {
      method: 'POST',
      headers: { 'Authorization': `Bearer ${token}` },
      body: JSON.stringify({ id: ev.display_id })
    });

    if (!response.ok) {
      const errorText = await response.text();
      return {
        success: false,
        error: `HTTP ${response.status}`,
        details: errorText
      };
    }

    return {
      success: true,
      data: await response.json()
    };

  } catch (error) {
    return {
      success: false,
      error: error.message,
      stack: error.stack
    };
  }
}
```

Retry Logic

```
import { credentialVaultClient } from '@dynatrace-sdk/client-classic-environment-v2';

// Retries network errors and 5xx responses only. A 4xx fails the same way on every
// attempt, so it is thrown at once instead of being sent again.
async function fetchWithRetry(url, options, maxRetries = 3) {
  for (let attempt = 1; attempt &lt;= maxRetries; attempt++) {
    let response;
    try {
      response = await fetch(url, options);
    } catch (error) {
      // Network error: retry unless this was the last attempt
      if (attempt === maxRetries) throw error;
      await new Promise(r =&gt; setTimeout(r, 1000 * attempt));
      continue;
    }

    if (response.ok) {
      return response;
    }

    if (response.status &gt;= 500 &amp;&amp; attempt &lt; maxRetries) {
      await new Promise(r =&gt; setTimeout(r, 1000 * attempt));
      continue;
    }

    throw new Error(`HTTP ${response.status}: ${await response.text()}`);
  }
  throw new Error('fetchWithRetry: maxRetries must be at least 1');
}

export default async function () {
  const token = (await credentialVaultClient.getCredentialsDetails({ id: 'CREDENTIALS_VAULT-
XXXXXXXXXXX' })).token;
  const response = await fetchWithRetry(
    'https://api.example.com/action',
    {
      method: 'POST',
      headers: { 'Authorization': `Bearer ${token}` }
    }
  );

  return await response.json();
}
```

Retry a `POST` only if the target treats a repeated request as a no-op. A 5xx can arrive after the target already acted, and the retry then creates the ticket, or runs the remediation, a second time.

5. Workflow-Level Failure Branching

`try` / `catch` inside a single JavaScript task handles **recoverable** errors — a transient API failure you can retry, a missing field you can default. **Workflow-level failure branching** handles the case where an entire

task has failed and you want to route around it: run a cleanup, send a failure notification, or skip downstream work.

The mechanism is the task `conditions` block. Each task can declare which predecessor states must hold for it to run, and what to do when the conditions are not met.

Predecessor-state conditions

Every task chained from a predecessor selects a state condition. The five values exposed in the workflow editor are:

State condition (editor)	Value in YAML / API	Run task when predecessor…
`success or skipped`	`OK`	…succeeded **or** was skipped by upstream branching (default)
`success`	`SUCCESS`	…succeeded only
`error or cancelled`	`NOK`	…failed **or** was cancelled
`error`	`ERROR`	…failed only — use for on-failure tasks
`any`	`ANY`	…finished with any outcome

The Automation API accepts exactly the five values `SUCCESS`, `ERROR`, `ANY`, `OK` and `NOK`, and `else` takes `SKIP` or `STOP`. The pairing with the editor labels follows their names and the Dynatrace workflow samples, which annotate `previous_task: OK` as *"Wait for previous_task to complete successfully"*.

A task waits for all of its predecessors

State conditions on several predecessors are combined with AND: *"The custom condition of a task is only checked if all the predecessor tasks are finished and fulfilled their state condition."* A failure task with several predecessors in a chain therefore never fires. If the first task fails, the later ones never run, so their conditions are never met. Give each step its own failure task with a single predecessor, which is the pattern the docs use: *"This task will only run if the run_tests task ends with an error."*

> **Sources:** Build workflows (DT docs) — the page lists the five state-condition options verbatim and shows reading them as a sentence ("Run this task if <predecessor> ended with <state>"); client-automation SDK — TaskConditionOptionStates (Dynatrace Developer); Dynatrace workflow samples (Dynatrace GitHub).

"Else" behaviour

When a task's conditions are not met, the **else** behaviour decides what happens next:

- **Skip** — mark this task as skipped; downstream tasks evaluate their own conditions (a downstream task with `success or skipped` still runs; one with `success` will not).
- **Stop** — no more tasks run on this branch of the workflow graph. This is the default.

Custom conditions

For checks beyond predecessor state — inspecting a field on a prior task's result — use a **custom** condition expressed as a Jinja expression that must evaluate to a boolean:

```
{{ result("problem_details")["problem"]["riskAssessment"]["riskScore"] &gt;= 9 }}
```

The `result("task_name")` function returns the result object of a previously executed task; `event()` and `execution()` are also available. See Workflow reference / Jinja expressions (DT docs) for the full expression catalog.

Task YAML

The shape below is the exported workflow format: `tasks` is a map keyed by task name, and each task's `conditions` holds `states`, an optional `custom` expression and `else`. If you generate workflows programmatically, export one you built in the editor and compare.

```
tasks:
  transform_payload:
    name: transform_payload
    action: dynatrace.automations:run-javascript
    predecessors:
      - fetch_errors
    conditions:
      states:
        fetch_errors: SUCCESS
    input:
      script: |
        // ... transform the query result

  notify_fetch_failed:
    name: notify_fetch_failed
    action: dynatrace.automations:run-javascript
    description: Notify on-call when fetch_errors fails
    predecessors:
      - fetch_errors
    conditions:
      states:
        fetch_errors: NOK
      else: SKIP
    input:
      script: |
        export default async function () {
          return { posted_failure_notice: 'fetch_errors' };
        }

  # notify_transform_failed and notify_post_failed have the same shape, each with a
  # single predecessor (transform_payload, post_to_webhook) set to NOK
```

Worked example — DQL → JS → webhook with a failure-notify branch

A common shape: query data, transform it, send it somewhere, and send a separate notice if any step fails.

Task	Type	Predecessor	State condition	Else
`fetch_errors`	DQL query	— (first task)	—	—
`transform_payload`	JavaScript	`fetch_errors`	`SUCCESS`	Stop (default)
`post_to_webhook`	HTTP request	`transform_payload`	`SUCCESS`	Stop (default)
`notify_fetch_failed`	JavaScript or HTTP	`fetch_errors`	`NOK`	Skip
`notify_transform_failed`	JavaScript or HTTP	`transform_payload`	`NOK`	Skip
`notify_post_failed`	JavaScript or HTTP	`post_to_webhook`	`NOK`	Skip

- **`fetch_errors`** runs a DQL query against logs or spans.
- **`transform_payload`** runs only if the query succeeded; **`post_to_webhook`** runs only if the transform succeeded.
- **Each `notify_*_failed` task** watches one step. Whichever step fails, its own failure task runs; the others do not. A single notify task with all three steps as predecessors would never run: when `fetch_errors` fails, the other two never run, so their conditions are never met.

In community practice, the failure-notify branch is kept narrowly scoped — a single notification or a quick cleanup — rather than a parallel happy-path. If recovery logic itself can fail, give it its own failure task as well.

Decision guidance — JS try/catch vs workflow-level branching

Dynatrace documents the `conditions` mechanism, not a rule for when to prefer it over `try` / `catch`. In community practice, the split below is how teams choose:

Use try/catch inside a JS task when…	Use workflow-level `conditions` when…
The error is recoverable — retry, default value, skip a missing field	The error is terminal for this task — downstream work cannot proceed
The decision logic is localized to one API call or one parse step	The decision spans multiple tasks (DQL query, transform, post)
You want the task to **succeed with a degraded result**	You want the task to **fail and route around it**
The error category is predictable in code (network timeout, JSON parse)	The error could come from any of several task types (DQL, HTTP, JS, action)

The two patterns compose. A JS task can `try/catch` its own recoverable errors and still throw (re-throw or `throw new Error(...)`) when the failure is genuinely terminal — at which point the workflow's `conditions` graph takes over and routes to the on-failure branch.

Related notebooks

- **WFLOW-07: Problem-Triggered Remediation** — remediation is a common use for failure conditions. A diagnose task runs; the remediation task runs only if the diagnosis succeeded; a verify task confirms the remediation; and the remediation and verify tasks each get their own failure task set to `NOK`.
- **WFLOW-05: PagerDuty & ServiceNow Integration** — PagerDuty/ServiceNow tasks are commonly chained as on-failure branches off a primary workflow — incident creation runs only if the primary path errored out.

> **Sources:** Build workflows (DT docs) — *"Stop means no more tasks are executed on this branch of the workflow graph."*, Workflow reference / Jinja expressions (DT docs).

6. Working with Data

Transform DQL Results

Group errors by service on spans, not logs. On the validation tenant (10/06/2026), `dt.service.name` was set on every span in a 15-minute sample and on none of 116,843 error logs in an hour; `dt.entity.service` was on 236 of them. Error logs group by `dt.source_entity` or `dt.entity.host` instead.

```
import { queryExecutionClient } from '@dynatrace-sdk/client-query';

// runQuery() as defined in §2

export default async function () {
  // Failed spans carry the service name; most error logs do not
  const records = await runQuery(`
    fetch spans, from: now() - 1h
    | filter span.status_code == "error"
    | fields start_time, dt.service.name, span.name
    | limit 100
  `);

  // Group by service
  const byService = records.reduce((acc, span) =&gt; {
    const service = span['dt.service.name'] || 'unknown';
    if (!acc[service]) {
      acc[service] = [];
    }
    acc[service].push(span['span.name']);
    return acc;
  }, {});

  // Create summary
  const summary = Object.entries(byService).map(([service, spans]) =&gt; ({
    service,
    error_count: spans.length,
    sample: spans[0]
  }));

  return {
    total_errors: records.length,
    by_service: summary
  };
}
```

Parse Event Tags

```
import { execution } from '@dynatrace-sdk/automation-utils';

export default async function () {
  const ev = (await execution()).params.event;   // trigger payload
  // "key:value" strings. entity_tags is deprecated in favor of primary_tags.*;
  // it may arrive as an array or as a JSON string.
  let tags = ev.entity_tags || [];
  if (typeof tags === 'string') {
    try { tags = JSON.parse(tags); } catch { tags = [tags]; }
  }

  // Parse key:value tags (split on the first colon only)
  const tagMap = tags.reduce((acc, tag) =&gt; {
    const i = tag.indexOf(':');
    if (i &gt; 0) {
      acc[tag.slice(0, i)] = tag.slice(i + 1);
    } else {
      acc[tag] = true;
    }
    return acc;
  }, {});

  return {
    team: tagMap['team'] || 'unknown',
    env: tagMap['env'] || 'unknown',
    is_critical: tagMap['critical'] || false
  };
}
```

Build Dynamic Queries

```
import { execution } from '@dynatrace-sdk/automation-utils';
import { queryExecutionClient } from '@dynatrace-sdk/client-query';

export default async function () {
  const ev = (await execution()).params.event;
  const entityId = ev.root_cause_entity_id;
  if (!entityId) {
    return { entity_type: null, errors: null, skipped: 'problem has no root-cause entity' };
  }

  // Build query based on entity type
  let query;
  if (entityId.startsWith('SERVICE-')) {
    query = `
      fetch spans, from: now() - 1h
      | filter dt.entity.service == "${entityId}"
      | filter http.response.status_code &gt;= 500
      | summarize errors = count()
    `;
  } else if (entityId.startsWith('HOST-')) {
    query = `
      fetch logs, from: now() - 1h
      | filter dt.entity.host == "${entityId}"
      | filter status == "ERROR"
      | summarize errors = count()
    `;
  } else {
    return { entity_type: entityId.split('-')[0], errors: null, skipped: 'no query for this
entity type' };
  }

  const records = await runQuery(query);   // runQuery() as defined in §2

  return {
    entity_type: entityId.split('-')[0],
    errors: records[0]?.errors ?? 0
  };
}
```

7. Passing Data Between Tasks

A workflow is a graph of tasks. Each task produces a result; downstream tasks read those results. Two access patterns — Jinja expressions for declarative task inputs, and the `result()` SDK function for JavaScript tasks — share the same underlying payload shape but differ in invocation.

7.1. The `result("task_name")` expression

Context	Form	Notes
Jinja (in any non-JS task's `input`)	`{{ result("task_name") }}`	Synchronous. Returns the task's full result object. Attribute access (`result("t").foo`) or `.get('foo', 'fallback')` for safer reads.
JavaScript task	`await result('task_name')`	**`result` is async in JS — forgetting `await` returns a Promise, not the data.** Imported from `@dynatrace-sdk/automation-utils`.

The returned payload shape is the same in both contexts; only the invocation differs.

> **Sources:** Workflow expression reference (DT docs), Run JavaScript Workflow Action (DT docs), Automation Utils SDK (Dynatrace Developer).

7.2. Result shape by task type

The accessor path depends on what the upstream task returned:

Upstream task type	Access path	Example
DQL query (`dynatrace.automations:execute-dql-query`)	`.records` — array of row objects	`{{ result("query_errors").records[0].error_count }}`
Run JavaScript (`dynatrace.automations:run-javascript`)	The returned object directly at top level	`{{ result("transform").affected_services }}`
HTTP request (`dynatrace.automations:http-function`)	`.status_code` for status, `.body` for the raw response body, `.json` for a parsed JSON response	`{{ result("post_webhook").status_code }}`

> **The `.output` wrapper is not universal.** Some older notebook patterns reference `result("task").output` — that field only exists when the upstream task literally returns an object containing a key named `output` (e.g., a JS task whose function returns `{ output: ... }`). It is not a built-in wrapper. Verify against the specific task type's documented result shape rather than assuming `.output` is always present.

7.3. Worked example — DQL → JS transform → notification

A common three-task pattern: query a metric, transform the result into a notification-friendly shape, send it.

Task 1 — `fetch_errors` (DQL):

```
name: fetch_errors
action: dynatrace.automations:execute-dql-query
input:
  query: |
    fetch spans, from: now() - 15m
    | filter span.status_code == "error"
    | summarize error_count = count(), by: { dt.service.name }
    | sort error_count desc
    | limit 5
```

Result shape: `{ records: [ { "dt.service.name": "checkout", error_count: 42 }, ... ] }`

Task 2 — `transform` (JavaScript), reads `fetch_errors`:

```
import { result } from '@dynatrace-sdk/automation-utils';

export default async function () {
  const upstream = await result('fetch_errors');
  const rows = upstream.records ?? [];

  if (rows.length === 0) {
    return { has_errors: false, summary: 'No failed requests in the last 15 minutes.' };
  }

  const top = rows[0];
  const summary_lines = rows.map(
    (r) =&gt; `- ${r['dt.service.name']}: ${r.error_count} errors`
  );

  return {
    has_errors: true,
    top_service: top['dt.service.name'],
    top_count: top.error_count,
    affected_count: rows.length,
    summary: `Top offender: ${top['dt.service.name']} (${top.error_count} errors)`,
    summary_lines,
  };
}
```

Result shape: `{ has_errors: true, top_service: "checkout", top_count: 42, affected_count: 3, summary: "...", summary_lines: [...] }`

Task 3 — `notify_slack` (Slack message), reads `transform`:

```
name: notify_slack
action: dynatrace.slack:slack-send-message
predecessors:
  - transform
conditions:
  states:
    transform: SUCCESS
  custom: '{{ result("transform").has_errors }}'
  else: SKIP
input:
  connection: slack-production      # the Slack connection ID
  channel: "#sre-alerts"
  message: |
    *Error spike detected*
    {{ result("transform").summary }}

    Affected services ({{ result("transform").affected_count }}):
    {% for line in result("transform").summary_lines %}{{ line }}
    {% endfor %}
```

The `conditions.custom` Jinja expression gates the notification: if `transform` returned `has_errors: false`, this task is skipped. Both settings matter. Without the `transform` predecessor, the task runs straight after the trigger, before `transform` has a result; without `else: SKIP`, an unmet condition stops the branch instead (Stop is the default). The Slack action's `connection` input is required.

> **Sources:** Build workflows (DT docs) — *"As a direct successor of the trigger, no task can define a state condition to a predecessor, as none exists."*; Slack Connector actions (DT docs).

7.4. Object passing — arrays, nested objects, flatten vs nested

JSON-serializable values pass cleanly between tasks: primitives, arrays, plain objects, nested combinations. There is no special encoding step.

When to keep nested:
- Downstream JS tasks consume the data (object access is ergonomic).
- The shape is logically grouped — `{ problem: { id, severity }, entities: [...] }` is clearer than ten flat fields.

When to flatten:
- Downstream Jinja templates read the data — deep paths like `{{ result("t").problem.entities[0].tags[2] }}` are fragile and unreadable.
- The notification channel has flat-key templating (e.g., a webhook expecting `entity_id` not `entities[0].id`).

Rule of thumb (community practice, not documented guidance): flatten at the **producer** side (in the JS transform), not at the consumer side. A transform task that emits `{ top_service, top_count, summary }` is easier for a downstream notification template to consume than one that emits `{ raw_records: [...] }`.

Size budget: task results are passed through the workflow execution store and shown in run history. Returning multi-megabyte payloads inflates run history and can hit the task-result size limit — for JavaScript

tasks, *"The task result size is limited to 6 MB."* If a DQL query returns 10,000 rows, summarize in the JS task (top 5, counts, p95) — do not pass the full record set downstream.

7.5. Variable scoping — what's available where

Source	Jinja	JavaScript	Scope
Trigger event	`{{ event() }}`, `{{ event()['display_id'] }}`	`const ex = await execution(); ex.params.event`	Workflow-wide — every task reads the same event
Previous task result	`{{ result("task_name") }}`	`await result('task_name')`	Workflow-wide once the task has completed
Workflow inputs	`{{ input('<name>') }}`	`(await execution()).input.<name>`	Workflow-wide; set at run time (API `input`) or from the workflow defaults
Secrets	Not in expressions — use a connection, or the action's Credential Vault field	`await credentialVaultClient.getCredentialsDetails({ id })`	Resolved at task execution; never return them in a result
Local variables inside one task	(not exported)	`const x = ...` inside `export default async function() { ... }`	Task-local — **not visible to other tasks unless returned**

A task's `return` value is the **only** way to make data visible downstream. There is no shared mutable state between tasks — every cross-task value travels through `result()`.

7.6. Common pitfalls

The `await`, bracket-access and `default` / `.get()` rows follow the cited expression reference and JavaScript action pages; the remaining rows reflect community practice — confirm them in your tenant.

Pitfall	Symptom	Fix
Forgetting `await` on `result()` in JS	Code gets a `Promise<Result>`, not the data; `.records` is `undefined`	Always `await result('task')`
Assuming `.output` is universal	`result("dql_task").output` returns `undefined`	Use `.records` for DQL, top-level fields for JS, `.body` / `.status_code` for HTTP
Hyphenated/dotted event keys in attribute syntax	`event().security-problem.technology` is parsed as subtraction	Use bracket form: `event()['security-problem.technology']`
`default` filter with missing parent	`result("t").foo | default("x")` still errors if `result("t")` itself is undefined	Use `.get()`: `result("t").get("foo", "x")`
Reading a task that may have errored	Downstream task fails with `Undefined variables`	Add a `conditions.states` gate (`success` only) or check `.get()` with a fallback
Reading a skipped task's result	A task gated by an unmet condition is *skipped*, not *failed*; `result("skipped_task")` may return an empty / undefined result	Branch the downstream task's `conditions` on the same predicate, not on the skipped task's result
Returning non-JSON-serializable values from JS (e.g., a `Date`, a class instance)	Silent coercion to string or `{}`	Return plain objects: `new Date().toISOString()`, plain literals
Returning huge payloads	Slow run history, possible truncation	Summarize in the producer task; return only what downstream tasks need

7.7. When this pattern doesn't fit

For long-running multi-step orchestrations where intermediate state must persist *outside* a single workflow run (resumable workflows, cross-workflow handoffs), workflow `result()` is the wrong tool — it lives only for the duration of one execution. Persistent state belongs in **Document Service** (for structured documents) or **Settings 2.0** (for configuration-shaped state).

> **Sources:** Workflow expression reference (DT docs), Run JavaScript Workflow Action (DT docs), Automation Utils SDK (Dynatrace Developer)., HTTP request action (DT docs) — result `status_code`, *"the HTTP response status code"*.
> **Derived:** the §7.2 result-shape table combines the `records` loop example in the expression reference, the JavaScript action's return-value section and the HTTP action's result fields.

8. Integration Examples

GitHub Issue Creation

```
import { execution } from '@dynatrace-sdk/automation-utils';
import { credentialVaultClient } from '@dynatrace-sdk/client-classic-environment-v2';

const GITHUB_REPO = 'my-org/my-repo';
const ENV_URL = 'https://&lt;your-environment-id&gt;.apps.dynatrace.com';

export default async function () {
  const ev = (await execution()).params.event;   // trigger payload
  const token = (await credentialVaultClient.getCredentialsDetails({ id: 'CREDENTIALS_VAULT-
XXXXXXXXXXX' })).token;
  // problem_link() is Jinja-only; in JS build the link from the problem ID
  const problemUrl = `${ENV_URL}/ui/apps/dynatrace.davis.problems/problem/${ev['event.id']}`;
  const response = await fetch(
    `https://api.github.com/repos/${GITHUB_REPO}/issues`,
    {
      method: 'POST',
      headers: {
        'Authorization': `token ${token}`,
        'Accept': 'application/vnd.github.v3+json',
        'Content-Type': 'application/json'
      },
      body: JSON.stringify({
        title: `[Dynatrace] ${ev['event.name']}`,
        body: `
## Problem Details
- **Problem ID:** ${ev.display_id}
- **Category:** ${ev['event.category']}
- **Started:** ${ev['event.start']}
- **Link:** [View in Dynatrace](${problemUrl})

## Affected Entities
${ev.affected_entity_ids.map(e =&gt; `- ${e}`).join('\n')}
        `,
        labels: ['dynatrace', ev['event.category'].toLowerCase()]
      })
    }
  );

  const issue = await response.json();
  return { issue_number: issue.number, url: issue.html_url };
}
```

Datadog Event

```
import { execution } from '@dynatrace-sdk/automation-utils';
import { credentialVaultClient } from '@dynatrace-sdk/client-classic-environment-v2';

export default async function () {
  const ev = (await execution()).params.event;   // trigger payload
  const token = (await credentialVaultClient.getCredentialsDetails({ id: 'CREDENTIALS_VAULT-
XXXXXXXXXXX' })).token;
  const response = await fetch('https://api.datadoghq.com/api/v1/events', {
    method: 'POST',
    headers: {
      'DD-API-KEY': token,
      'Content-Type': 'application/json'
    },
    body: JSON.stringify({
      title: ev['event.name'],
      text: `Problem ${ev.display_id} detected by Dynatrace`,
      alert_type: ev['event.category'] === 'AVAILABILITY' ? 'error' : 'warning',
      source_type_name: 'dynatrace',
      tags: ['source:dynatrace', `category:${ev['event.category']}`]
    })
  });

  return await response.json();
}
```

Splunk HEC Event

```
import { execution } from '@dynatrace-sdk/automation-utils';
import { credentialVaultClient } from '@dynatrace-sdk/client-classic-environment-v2';

const SPLUNK_HEC_URL = 'https://splunk.example.com:8088';

export default async function () {
  const ev = (await execution()).params.event;   // trigger payload
  const token = (await credentialVaultClient.getCredentialsDetails({ id: 'CREDENTIALS_VAULT-
XXXXXXXXXXX' })).token;
  const response = await fetch(
    `${SPLUNK_HEC_URL}/services/collector/event`,
    {
      method: 'POST',
      headers: {
        'Authorization': `Splunk ${token}`,
        'Content-Type': 'application/json'
      },
      body: JSON.stringify({
        event: {
          problem_id: ev.display_id,
          title: ev['event.name'],
          category: ev['event.category'],
          affected_entities: ev.affected_entity_ids
        },
        sourcetype: 'dynatrace:problem',
        index: 'observability'
      })
    }
  );

  return { sent: response.ok };
}
```

9. Performance Tips

Parallel Execution

```
import { execution } from '@dynatrace-sdk/automation-utils';

export default async function () {
  const entityIds = (await execution()).params.event.affected_entity_ids;

  // Execute all lookups in parallel.
  // placeholder: getEntityDetails() is yours, e.g. a monitoredEntitiesClient.getEntity() call
(§2)
  const results = await Promise.all(
    entityIds.map(id =&gt; getEntityDetails(id))
  );

  return { entities: results };
}
```

Limit Query Scope

```
// runQuery() as defined in §2

// Bad: seven days of every ERROR log, every field
const everything = await runQuery(`fetch logs, from: now() - 7d | filter status == "ERROR"`);

// Good: limited time range and fields
const recent = await runQuery(`
  fetch logs, from: now() - 1h
  | filter status == "ERROR"
  | fields timestamp, content
  | limit 100
`);
```

Use Timeouts

```
// Add timeout to external calls
const controller = new AbortController();
const timeout = setTimeout(() =&gt; controller.abort(), 10000);

try {
  const response = await fetch(url, {
    signal: controller.signal,
    // ... other options
  });
  return await response.json();
} finally {
  clearTimeout(timeout);
}
```

10. Configuring Task Timeouts

Two timeouts apply to every workflow task — see WFLOW-01 § *Platform Limits* for the conceptual split between **task timeout** (wall-clock budget for the whole task) and **Dynatrace runtime timeout** (per-action budget, 120s default, applies to DQL queries and individual JS/HTTP actions). This section covers the configuration shape on each of the three task types most authors touch.

`timeout` field — units and shape

The `timeout` field on a workflow task is the **task timeout**. It accepts an integer count of **seconds** when configured in JSON/YAML via the AutomationEngine. The Workflows UI exposes it under **Options → Adapt timeout** and labels the field *Timeout this task (seconds)*. Default: 60 minutes (3600s). Maximum: 7 days (604800s).

Waiting is a task option, not a separate action. **Wait before** (`waitBefore`) pauses a task before it runs, from 0 to 86,400 seconds. An approval task such as Slack **Request approval** waits for an answer until the timeout set in its **Options** tab passes. Both are integer seconds, or a Jinja expression. See the approval example below.

On `execute-dql-query`

```
tasks:
  query_errors:
    name: query_errors
    description: Count error logs in the last hour
    action: dynatrace.automations:execute-dql-query
    timeout: 180   # task timeout, seconds — caps the whole task
    input:
      query: |
        fetch logs, from: now() - 1h
        | filter status == "ERROR"
        | summarize errors = count()
      # Note: the DQL engine's own 120s runtime budget still applies
      # to the query execution itself. Raising `timeout` above 120s
      # only helps if the task does more than execute one query.
```

On `run-javascript`

```
tasks:
  enrich_problem:
    name: enrich_problem
    action: dynatrace.automations:run-javascript
    timeout: 300   # task timeout, seconds
    input:
      script: |
        import { queryExecutionClient } from '@dynatrace-sdk/client-query';
        export default async function() {
          const started = await queryExecutionClient.queryExecute({
            body: {
              query: `fetch logs, from: now() - 1h | filter status == "ERROR" | summarize
errors = count()`,
              // How long queryExecute waits for the result before returning a
              // requestToken instead, in milliseconds. Distinct from both the
              // task timeout above and the 120s runtime budget.
              requestTimeoutMilliseconds: 30000
            }
          });
          let response = started;
          while (response &amp;&amp; (response.state === 'NOT_STARTED' || response.state ===
'RUNNING')) {
            response = await queryExecutionClient.queryPoll({
              requestToken: started.requestToken,
              requestTimeoutMilliseconds: 30000
            });
          }
          if (!response || response.state !== 'SUCCEEDED') {
            throw new Error(`DQL query did not succeed: ${response?.state ?? 'no response'}`);
          }
          return { errors: response.result.records[0]?.errors ?? 0 };
        }
```

Inside a JS task you actually deal with **three** timeouts: the task `timeout`, the runtime's 120s per-action ceiling, and any `requestTimeoutMilliseconds` you pass to SDK calls. The first one budgets the task as a whole; the second one is enforced by the runtime regardless; the third one is yours to set per outbound call.

On approval and delayed tasks

Approval tasks intentionally wait for a human signal, so their timeouts are long by design. Slack **Request approval** has no timeout input of its own: the docs say to configure it in the action's **Options** tab, where the task timeout (**Adapt timeout**) is set. Confirm the window with a test run (WFLOW-07 §7). A task that only needs to start later uses `waitBefore`:

```
tasks:
  wait_for_approval:
    name: wait_for_approval
    action: dynatrace.slack:request-approval
    timeout: 1800        # the approval window: 30 minutes, in seconds
    input:
      connection: slack-production
      channel: "#approvals"
      message: "Approve restarting {{ event().get('root_cause_entity_id', 'the affected
component') }}?"

  recheck_problem:
    name: recheck_problem
    action: dynatrace.automations:run-javascript
    waitBefore: 900      # stay waiting 15 minutes before running; max 86400
    input:
      script: |
        // re-query the problem state (WFLOW-04 §6)
```

See **WFLOW-07 § Approval Workflows** for the full human-in-the-loop pattern.

Decision guidance: raise the timeout, or fix the query?

When a DQL or JavaScript task hits a timeout, the right move depends on **which** timeout fired:

Symptom	What hit	First move
Task fails at ~120s with a runtime/engine error	Dynatrace runtime timeout (per-action)	Narrow the query window (`from: now() - 15m` instead of `now() - 24h`), pre-aggregate into a metric or bizevent, or split into multiple smaller tasks. Raising the task `timeout` will not help.
Task fails at exactly the configured task `timeout` value	Task timeout	Raise `timeout` , or break the task into smaller tasks. Verify the work genuinely needs the budget — most legitimately-long tasks are waiting on a human or external system, not computing.
`queryExecute()` returns well before 120s with no `result` — only the query state and a request token	The query outlasted `requestTimeoutMilliseconds`	Poll with `queryPoll` using the returned token — the §2 `runQuery()` helper does this. Raising `requestTimeoutMilliseconds` only moves the point where polling starts; code that reads `result.result.records` without checking `state` breaks either way.
Approval task ends at its `timeout` value	Task timeout on the approval task	This is usually the correct behavior — the human did not respond. Decide whether to escalate, auto-approve, or fail.

> **Rule of thumb.** Raise timeouts only after narrowing scope. A task that needs 10 minutes of DQL is almost always a task that needs a pre-aggregation upstream — fix that first.

> **Sources:** Build workflows — Adapt timeout (DT docs), Run JavaScript action (DT docs), Grail DQL query API — client-query (Dynatrace Developer) — *"If the query succeeded, the result will be included. Otherwise the response will contain a request token to reference the query in future polling requests."*; `queryPoll` : *"Retrieves query status and final result from Grail."* Build workflows — Wait before (DT docs) — *"The Wait before option controls how long a task stays in the waiting state before being run."*; Slack Connector actions (DT docs) — *"Configure the timeout in the action's Options tab."* **Derived:** the three-timeout interaction model in *On* `run-javascript` and the decision table combine the cited per-field documentation; no single source presents them together.

Monitor JavaScript Task Performance

```
// JavaScript task execution times
// Data object corrected 08/12/2026. Workflow executions are NOT in `events`, and there is no
// `automation.task.execution` / `automation.workflow.execution` event type in any spelling — those
// filters matched nothing, silently, in 25 cells. AutomationEngine writes to `dt.system.events` with
// `event.kind == "WORKFLOW_EVENT"` (5.7M records / 30d here), split by `event.type`:
//   WORKFLOW_EXECUTION · TASK_EXECUTION · ACTION_EXECUTION · WORKFLOW_CREATED/UPDATED/DELETED
// Fields are the `dt.automation_engine.*` family:
//   .workflow.title / .workflow.id      (was workflow.name)
//   .task.name                          (was task.name)
//   .action.function / .action.app      (was task.type — e.g. run-javascript, send-email,
//                                        snow-search-incidents)
//   .state                              (was task.status / execution.status)
//                                        values RUNNING · SUCCESS · ERROR · DISCARDED · SKIPPED
//                                        — note "FAILED" is NOT a value; it is ERROR
//   .state.is_final                     true only on terminal records — filter on it, otherwise a
//                                        single execution is counted once as RUNNING and again as
//                                        SUCCESS/ERROR
//   .state_info                         (was task.error)  ·  duration (was task.duration)
// Enumerate with:
//   fetch dt.system.events, from:-24h | filter event.kind == "WORKFLOW_EVENT" | limit 1
fetch dt.system.events, from:-7d
| filter event.kind == "WORKFLOW_EVENT"
| filter event.type == "ACTION_EXECUTION"
| filter dt.automation_engine.state.is_final == true
| filter dt.automation_engine.action.function == "run-javascript"
| summarize {executions = count(), avg_ms = avg(duration) / 1ms, p95_ms = percentile(duration, 95) / 1ms}, by:{dt.automation_engine.workflow.title}
| sort p95_ms desc
| limit 20
```

```
// An empty result here is the HEALTHY answer: on the validation tenant (10/06/2026) http-
function
// ran 70 times in 7 days, all SUCCESS. Rows appearing means real HTTP task failures.
// HTTP request task errors
// Data object corrected 08/12/2026. Workflow executions are NOT in `events`, and there is no
// `automation.task.execution` / `automation.workflow.execution` event type in any spelling —
those
// filters matched nothing, silently, in 25 cells. AutomationEngine writes to
`dt.system.events` with
// `event.kind == "WORKFLOW_EVENT"` (5.7M records / 30d here), split by `event.type`:
//   WORKFLOW_EXECUTION · TASK_EXECUTION · ACTION_EXECUTION · WORKFLOW_CREATED/UPDATED/DELETED
// Fields are the `dt.automation_engine.*` family:
//   .workflow.title / .workflow.id      (was workflow.name)
//   .task.name                          (was task.name)
//   .action.function / .action.app      (was task.type — e.g. run-javascript, send-email,
//                                        snow-search-incidents)
//   .state                              (was task.status / execution.status)
//                                        values RUNNING · SUCCESS · ERROR · DISCARDED ·
SKIPPED
//                                        — note "FAILED" is NOT a value; it is ERROR
//   .state.is_final                    true only on terminal records — filter on it,
otherwise a
//                                        single execution is counted once as RUNNING and again
as
//                                        SUCCESS/ERROR
//   .state_info                         (was task.error)  ·  duration (was task.duration)
// Enumerate with:
//   fetch dt.system.events, from:-24h | filter event.kind == "WORKFLOW_EVENT" | limit 1
fetch dt.system.events, from:-24h
| filter event.kind == "WORKFLOW_EVENT"
| filter event.type == "ACTION_EXECUTION"
| filter dt.automation_engine.action.function == "http-function"
| filter dt.automation_engine.state == "ERROR"
| fields timestamp, dt.automation_engine.workflow.title, dt.automation_engine.state_info
| sort timestamp desc
| limit 25
```

11. CMDB-Driven Host Tag Enrichment

A frequent ask: read host→application mappings from a CMDB and apply them as host tags in bulk. This is the capstone that combines this notebook's building blocks — an SDK DQL query (§2), `fetch()` to a Dynatrace API with auth (§3), retry logic (§4), `result()` / `withItems` data passing between tasks (§7), and the Credential Vault — into one deployable workflow.

The shape is two `run-javascript` tasks:

1. **`query_hosts_and_enrich`** — enriches `dt.entity.host` through a CMDB lookup chain (`lookup [load "/lookups/…"]`), dedups by host id, and diffs against existing tags → returns `hosts[]` .
2. **`apply_tags_to_hosts`** — loops (`withItems`) over `hosts[]` and POSTs a `set hostTag` operation to the **OneAgent Remote Configuration Management API** (`/api/v2/oneagents/remoteConfigurationManagement`) as the workflow actor (relative URL, `fleet-management:oneagents:write` ; a classic `oneAgents.write` token from the Credential Vault for classic or

hybrid environments). Guardrails: `DRY_RUN`, `MAX_HOSTS`, skip-existing, 409-retry. (Config is supplied via workflow inputs.)

One `set hostTag` operation writes both `primary_tags.*` (the prefix is mandatory) and `dt.*` primary fields (`dt.security_context`, `dt.cost.costcenter`) — see **FAQ-02 (Tagging: Sources, Standards & Strategy)** for the tagging-source model.

> **Check first whether you need this workflow at all.** If the tag value can be derived from context the host already reports — host group, host name, an existing host tag — the Latest Dynatrace path is **Ingest enrichment configuration** (FAQ-02 § 3.3): a central rule, OneAgent 1.343+, with *"No changes on the hosts are required"* and *"No agent restart is needed."* This workflow is for the case a rule cannot express: a per-host value that lives only in an external CMDB.
>
> It also carries a restart cost the rule does not. The API reference states that *"By default OneAgents will be restarted when network zone, host group, host tags or host properties are reconfigured - the restart is required to apply the changes."* Plan live runs as restart events, or accept that tags apply only at each host's next restart.
>
> **Hands-on build:** the complete step-by-step walkthrough — build it in the Workflows editor, both scripts, customization, the loop/condition wiring, the safety model, and an import-ready YAML skeleton — is in the **WFLOW-95 LAB: CMDB-Driven Host Tag Enrichment**.
>
> **Sources:** OneAgent remote configuration management API — POST a configuration job (DT docs) — *"Required scope: fleet-management:oneagents:write"* (platform token / OAuth), read 09/28/2026, Lookup data in Grail (DT docs), OneAgent tag setup — Ingest enrichment configuration (DT docs) — *"No agent restart is needed."*

Next Steps

With custom integrations ready, learn governance:

Recommended Path

1. **WFLOW-09: Security, Governance & Monitoring** - Production best practices

Key Takeaways

- **JavaScript actions** enable custom logic
- **Dynatrace SDK** provides typed API access
- **HTTP requests** integrate any REST API whose host is in External requests (and behind EdgeConnect when private)
- **Error handling** is critical for reliability — inside a task use `try/catch`; between tasks use the workflow `conditions` block
- **On-failure branches** use `error` / `error or cancelled` state conditions (`ERROR` / `NOK` in YAML), one failure task per step
- **Performance** matters - limit query scope, use parallelism

- **Bulk host operations** combine an enrichment query with a `withItems` loop over the Remote Configuration Management API — introduced in §11, full hands-on build in the **WFLOW-95 LAB**

Summary

In this notebook, you learned:

- JavaScript action structure and context
- Dynatrace SDK clients and methods
- HTTP request patterns (GET, POST, auth)
- Error handling and retry logic
- Workflow-level failure branching with task `conditions` (predecessor state and custom Jinja)
- Data transformation patterns
- Integration examples (GitHub, Datadog, Splunk)
- Performance optimization tips
- A CMDB-driven host tag enrichment capstone (§11) — full build in the **WFLOW-95 LAB**

References

- Run JavaScript action (DT docs)
- HTTP request action (DT docs)
- Workflow reference / Jinja expressions (DT docs)
- Dynatrace Query Language reference (DT docs)
- OneAgent remote configuration management API — POST a configuration job (DT docs)
- Lookup data in Grail (DT docs)
- Dynatrace SDK (Dynatrace Developer)
- Dynatrace Developer Portal (Dynatrace)

This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.